INDUSTRIAL TRAINING REPORT

on

“Library Book Management:
A Python-Based Design”

by

Mayank Kumar [ROLL NUMBER TO FILL]

Submitted in partial fulfillment of the requirements for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

from

EWB EDUTECH PRIVATE LIMITED (EWB Courses)

Submitted to

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING & TECHNOLOGY

UNIVERSITY OF LUCKNOW, LUCKNOW

November 2026

DECLARATION

I, Mayank Kumar, declare that this report titled “Library Book Management: A Python-Based Design” is a theory-based academic case study prepared for the B.Tech CSE summer training submission. The concepts, illustrative records, diagrams and calculations are presented as a proposed design; no working software, deployment or empirical evaluation is claimed.

The attached certificate is the authentic student-supplied document evidencing completion of a one-month Python Programming internship. The certificate does not identify this library topic as the assigned internship project. The library scenario should be described as completed internship work only if supported by separate evidence.

The fictional book, copy, member and loan IDs in this report are used solely to explain the design. No confidential or restricted company information is disclosed. References and supplied documents are acknowledged at the end of the report.

Signature: _____

Name: Mayank Kumar

Roll No.: [TO FILL]

Section: [TO FILL]

Date: [TO FILL]

TRAINING / INTERNSHIP CERTIFICATE



Colour reproduction of the student-provided original certificate. Issued 15 July 2026; the document states a one-month Python Programming internship. The original PDF should be used for final printing if higher fidelity is required.

ABSTRACT

This report describes a proposed library book management system using beginner-level Python concepts. The case study addresses the difficulty of tracking physical book copies, registered members and issue-return transactions consistently in a small library. The focus is on process design, not on producing a working model.

The suggested data model separates a book title from its physical copies and records every borrowing event as a loan. The key rule is that one physical copy may have at most one active loan. Issue, return, search, validation and summary reporting are explained using flowcharts, fictional tables, manually worked examples and an illustrative chart.

The supplied internship certificate confirms one-month Python Programming training for Mayank Kumar. It does not name the library case study. Therefore, the present design is treated as an academic application of Python-related ideas rather than a verified project delivered to the training provider.

Keywords: Python, library management, catalogue, circulation, conceptual design, case study.

ACKNOWLEDGEMENT

I thank the Department of Computer Science and Engineering, Faculty of Engineering & Technology, University of Lucknow, for providing the format and opportunity to prepare a summer internship report.

I acknowledge EWB EDUTECH PRIVATE LIMITED / EWB Courses for the Python Programming internship documented by the attached completion certificate. The concepts discussed here are connected to the subject area of that training; the particular library case study is a separate academic design.

I also thank the faculty member reviewing this report for guidance and feedback. Any mistakes in the explanatory examples or calculations remain my responsibility.

Note to student before submission: revise this acknowledgement to reflect the people who actually supported your work.

TABLE OF CONTENTS

Declaration i
Training / Internship Certificate ii
Abstract iii
Acknowledgement iv
Table of Contents v
List of Tables vi
List of Figures vii
1. Introduction 1
2. Background and Motivation 2
3. Nature of the Training and Project 3
4. Objective of the Project 4
5. Scope and Boundaries 5
6. Overview of the Organisation 6
7. Overview of Python Training Relevance 7
8. Technology Overview: Core Concepts 8
9. Technology Overview: Data Structures 9
10. Technology Overview: Storage Options 10
11. Proposed Modules 11
12. Requirement Analysis 12
13. Problem Statement 13
14. Assumptions and Feasibility 14
15. Stakeholders and Use Cases 15
16. Overall System Design 16
17. Data Model and Relationships 17
18. Book and Copy Data Dictionary 18
19. Member and Loan Data Dictionary 19
20. Business Rules 20
21. Issue Workflow 21
22. Return Workflow 22
23. Search and Catalogue Workflow 23
24. Availability Calculation 24
25. Validation and Error Handling 25
26. Privacy and Basic Security 26
27. Illustrative Dataset 27
28. Case Study Scenario 28
29. Worked Example: First Issue 29
30. Worked Example: Duplicate Issue 30
31. Worked Example: Return 31
32. Worked Example: Search 32
33. Summary Reporting 33
34. Charts and Interpretation 34
35. Proposed Test Plan 35
36. Proposed Test Cases 36
37. Design Limitations 37
38. Future Scope 38
39. Conclusion 39
References 40
Appendix A: Glossary 41
Appendix B: Submission Checklist 42

LIST OF TABLES

Table 1. Evidence and limits of attribution. 3

Table 2. Boundary of the proposed design. 5

Table 3. Basic Python concepts and their proposed role. 8

Table 4. Conceptual storage comparison. 10

Table 5. Selected requirements. 12

Table 6. Explicit case-study assumptions. 14

Table 7. Example stakeholder roles. 15

Table 8. Book and copy fields; names are fictional. 18

Table 9. Member and loan fields; dates are illustrative. 19

Table 10. Invented availability example, not a measured library record. 24

Table 11. Example validation responses. 25

Table 12. Fictional catalogue for the worked example. 27

Table 13. Manually derived scenario totals. 28

Table 14. Predicted state change for the first issue. 29

Table 15. Predicted return state; example dates only. 31

Table 16. Expected search behaviour, not an application screenshot. 32

Table 17. Manually calculated example summary. 33

Table 18. Planned tests; none are claimed as executed. 36

LIST OF FIGURES

Figure 1. Conceptual sequence of core library activities. 2

Figure 2. General Python-oriented processing pattern. 7

Figure 3. Relationship between descriptive and transactional records. 9

Figure 4. Proposed high-level module arrangement. 11

Figure 5. Conceptual data flow; no application was implemented. 16

Figure 6. Simplified conceptual entity relationship chain. 17

Figure 7. Issue decision flow, shown at a conceptual level. 21

Figure 8. Return decision flow. 22

Figure 9. Proposed search process. 23

Figure 10. Duplicate-issue prevention in the worked example. 30

Figure 11. Illustrative available/issued counts; invented values. 34

Figure 12. Possible future development sequence, not work completed. 38

1. Introduction

A library makes books available to readers while keeping track of which copy is currently on a shelf and which copy is on loan. Even in a small library, manually recording issue dates, return dates and member details can lead to misplaced records and inconsistent availability information.

This report presents a conceptual design for a library book management system using Python. It is an academic case study, not a claim that a software product was built, installed or used by EWB Courses. The design combines simple rules, data tables, flowcharts and a worked example to explain how such a system could operate.

The proposed scope is intentionally limited to book cataloguing, member registration, issue, return, search and basic summary reports. This narrow scope makes the rules understandable for a beginner in Python programming.

- Training evidenced by the attached certificate: a one-month Python Programming internship.
- Report method: design analysis and illustrative examples only; no running model or measured deployment results.

2. Background and Motivation

A paper register is easy to start but becomes harder to search as entries accumulate. A librarian may need to check several pages before confirming whether a title is available or which member has borrowed a copy. A consistent digital record would reduce that lookup effort.

Python is a suitable teaching language for describing the proposed solution because it offers readable control flow, lists and dictionaries for small datasets, and established options for file or database storage. No specific library installation, dataset or organisational process is assumed here.

The design aims to make the movement of a book copy traceable: it starts as available, is issued to a registered member, and becomes available again when returned. The loan record preserves the history of that movement.



Figure 1. Conceptual sequence of core library activities.

3. Nature of the Training and Project

The supplied offer letter names EWB EDUTECH PRIVATE LIMITED, describes a virtual Python Programming Intern role and states a duration of one month. Its start date is marked “to be announced.” The supplied completion certificate awards Mayank Kumar a one-month internship in Python Programming and is dated 15 July 2026.

The certificate mentions an assigned real-world project but does not identify its topic. Accordingly, this library design is presented as a separate, illustrative report case study consistent with the subject of Python training. It should not be described as the internship project unless the student can confirm that it was.

Learning areas relevant to this case study include variables, conditional statements, loops, functions, structured records, validation and file-handling concepts. Any first-person claim of completing a specific exercise should be added only after checking personal training records.

Evidence	What it establishes
Offer letter	Organisation, role, virtual mode, one-month duration
Certificate	Name, Python Programming completion, issue date
This case study	A proposed academic design, not provider evidence

Table 1. Evidence and limits of attribution.

4. Objective of the Project

The overall objective is to specify a small, understandable library book management process that could later be implemented with beginner-level Python. The report emphasises clarity of data and rules rather than a graphical interface or advanced algorithms.

A successful conceptual design should answer four questions: what information is stored, what action a user performs, what condition makes the action valid, and how the stored state changes afterward. Each process is therefore described in terms of its input, validation, action and output.

- Define book, member and loan records with unique identifiers.
- Describe issue and return rules that prevent duplicate active loans of one copy.
- Specify a search procedure for books by title, author or category.
- Provide an illustrative dataset and manually worked outcomes.
- List test cases that a future implementation would need to pass.

No performance claim, software screenshot or implemented output is included.

5. Scope and Boundaries

This report assumes a single small library and one librarian maintaining the records. It is a design study; it does not cover equipment procurement, a production deployment or real borrower information. Example names and records are invented solely to demonstrate the rules.

The system boundary contains the catalogue, member register, loan register and summary calculations. Activities such as purchasing books or collecting actual fine payments remain outside the boundary. A theoretical late-return rule may be discussed without asserting any institution’s policy.

A physical copy must have a unique copy identifier even when several copies share the same book title. This distinction avoids the common error of treating a title as if it could be lent only once.

Included	Excluded
Book and copy records	Supplier procurement
Membership and loan status	Online payment processing
Search and simple summaries	Mobile app and barcode hardware
Illustrative tests	Live user testing or measured results

Table 2. Boundary of the proposed design.

6. Overview of the Organisation

The submitted offer letter identifies the training organisation as EWB EDUTECH PRIVATE LIMITED and describes the programme as a virtual internship. The supplied certificate carries EWB Courses branding and confirms completion of Python Programming training for the named student.

The offer letter says participants should attend sessions, complete assessments and assigned activities and follow programme instructions. Those statements describe programme expectations, not evidence of any specific library-system deliverable.

No internal business information, training materials, employee details or private datasets from the organisation are used in this report. The library setting, sample records and diagrams have been created only for the academic case study.

Source: student-supplied offer letter (14 June 2026) and completion certificate (15 July 2026).

7. Overview of Python Training Relevance

Python programs commonly organise tasks into small functions. For a hypothetical library system, functions could accept a book identifier, check a condition and update a record. This keeps issue, return and search processes separate and easier to review.

Lists are useful for a simple sequence of records; dictionaries let a program associate field names with values. Conditional statements express rules such as “issue only if the copy is available,” and loops can inspect each record for matches. These are conceptual applications of basic programming ideas, not evidence of submitted code.

A beginner-friendly design should first prioritise correct identifiers and state changes. A database, login system and interface can be considered later if a real implementation is authorised and required.



Figure 2. General Python-oriented processing pattern.

8. Technology Overview: Core Concepts

A variable stores a value such as copy ID or due date. A condition selects a valid path: for example, the issue operation proceeds only when the member exists and the chosen copy has no active loan. A loop checks a collection of book records when searching.

A function groups a task under a name and accepts inputs. Possible conceptual functions include find_copy, issue_copy, return_copy and list_overdue. A return value may indicate success or provide a reason for failure. This design does not prescribe actual function signatures.

Readable naming is important. “copy_id” describes a physical item; “book_id” identifies a title or edition. Confusing them can create incorrect availability counts when a title has multiple copies.

Python concept	Possible use in the case study
Condition	Reject issue when no copy is available
Loop	Scan matching catalogue entries
Function	Separate issue and return rules
Dictionary	Represent labelled fields in a small record

Table 3. Basic Python concepts and their proposed role.

9. Technology Overview: Data Structures

A book record contains bibliographic information shared by copies: title, author and category. A copy record identifies a particular physical item. A member record identifies a borrower. A loan record links a member to a copy for a defined issue and due date.

For a classroom example, a list of dictionaries can represent these records. With many books, repeated scans are inefficient; a dictionary keyed by unique ID or a relational database is more appropriate. The design can therefore begin simply without making an unsupported claim about scalability.

The choice of structure should reflect the question being asked. A list preserves a sequence; an ID-based mapping supports quick lookup; a table with foreign keys helps enforce relationships when using a database.

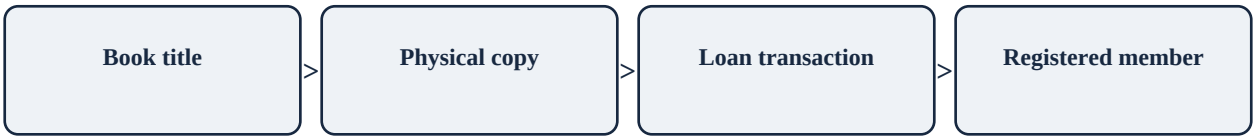


Figure 3. Relationship between descriptive and transactional records.

10. Technology Overview: Storage Options

The simplest prototype could keep records in memory while demonstrating operations. Such records vanish when the program ends, so they would not be adequate for real library service. A CSV file is easy to inspect but requires careful handling of related records and simultaneous edits.

A local relational database such as SQLite would be a reasonable future choice for a small single-location implementation. It supports structured tables and transactions. However, the present report stops at a conceptual schema; it does not claim that a database was created.

Any implementation must consider backups, access control and recovery before holding real member details. These responsibilities are separate from understanding the basic issue-return logic.

Option	Strength	Limitation
Memory only	Very simple to demonstrate	No persistence
CSV files	Human-readable export	Weak relational safeguards
SQLite	Structured tables and transactions	Needs database design and backup

Table 4. Conceptual storage comparison.

11. Proposed Modules

A modular design divides the system into areas that each have one responsibility. The catalogue module stores books and copies. The membership module stores permitted borrowers. The circulation module creates and closes loans. The reporting module presents counts from existing records.

The user interface, if later built, would call these modules rather than contain all validation itself. This separation lets the same rule be used for different interfaces, such as a command-line menu or a web screen. No interface is implemented in this report.

The reporting module must read from the same authoritative loan records as circulation. Maintaining separate manually updated “available” totals would risk discrepancies unless those totals are recalculated or updated atomically.

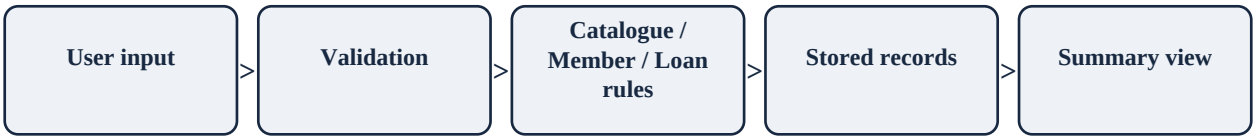


Figure 4. Proposed high-level module arrangement.

12. Requirement Analysis

Functional requirements describe actions the proposed design should support. Non-functional requirements describe qualities such as correctness, privacy and maintainability. Both are necessary: a quick search is not useful if a returned copy remains marked as borrowed.

Each requirement should be testable without guessing. “The system is good” is vague; “a copy with an active loan cannot be issued again” can be checked with a sample case. Because the report is theoretical, the tests listed later are planned checks, not executed test results.

ID	Requirement
F1	Register a book title and each physical copy
F2	Register a member with a unique member ID
F3	Issue only a currently available copy
F4	Close an active loan on return
F5	Search catalogue by text or category
N1	Protect real member data if implemented

Table 5. Selected requirements.

13. Problem Statement

A small library using paper records may find it difficult to determine which physical copies are available and which borrower currently holds a particular item. Duplicate or incomplete entries can make issue and return operations unreliable. The problem is to define a consistent method for representing books, copies, members and active loans.

The proposed design must prevent an already-issued copy from being issued again, while allowing two different copies of the same title to be lent independently. It must also preserve a clear record of when a copy was issued and returned.

The problem is deliberately limited to logical record handling. The report does not claim to solve hardware, staffing, procurement, institutional rules or actual deployment questions.

Design question: How can simple Python-oriented rules maintain an accurate copy-level loan status?

14. Assumptions and Feasibility

The illustrative library has a small catalogue, a single librarian and registered members. Each physical copy has a unique label. One copy can have at most one active loan; a member may have more than one loan only if a future local policy permits it.

From a technical standpoint the data and rules are simple enough to describe with core Python constructs. From an operational standpoint the librarian would need to enter accurate IDs and follow a return procedure. The project’s feasibility as a real service cannot be established without users, infrastructure and institutional approval.

The worked example later assumes a fourteen-day loan period solely for demonstration. It is not represented as the policy of the University of Lucknow or EWB Courses.

Assumption	Reason for the example
Unique copy ID	Distinguishes identical titles
Registered member ID	Links loan to borrower
14-day example period	Makes a due-date calculation visible
Single operator	Keeps process description simple

Table 6. Explicit case-study assumptions.

15. Stakeholders and Use Cases

The librarian is the operator who creates records and manages circulation. A member borrows and returns copies. A supervisor may review aggregate reports but should not need unrestricted access to personal details for simple counts.

In a conceptual use case, the librarian selects “issue,” enters the member ID and copy ID, checks the proposed due date and confirms the action. If either ID is unknown or the copy is already on loan, the action is rejected without changing records.

The essential use cases are catalogue entry, member registration, issue, return, search and summary viewing. All examples in this report assume that actions are performed by an authorised operator.

Actor	Typical permitted action
Librarian	Create records, issue and return
Member	Request a book or return a copy
Supervisor	Review non-personal totals

Table 7. Example stakeholder roles.

16. Overall System Design

The conceptual design has three layers. An input layer accepts book, copy and member identifiers. A rules layer checks the current records before an action. A storage layer would retain book, copy, member and loan entries. A later interface should show success or a specific validation error.

For example, the issue rule reads a copy and checks for any open loan for that copy. It then records an issue date and due date. The return rule looks up the active loan and adds a return date. Availability is derived from whether an active loan exists.

An explicit record of transactions is preferable to changing a single Boolean without history: the history can explain past borrowing and support later audit questions.



Figure 5. Conceptual data flow; no application was implemented.

17. Data Model and Relationships

The Book entity identifies a title or edition and has one or more Copy entities. Each Copy belongs to exactly one Book. A Loan associates one Copy and one Member with issue, due and optional return dates. A Member can have multiple historical loans.

The active-loan constraint is the central design rule: for a given copy ID, at most one loan may have no return date. This ensures that availability is a consequence of recorded transactions rather than an independently guessed value.

When adding a new title, the operator first creates its Book record and then labels each Copy. When retiring a copy, an open loan must be closed or investigated before deletion or archival.

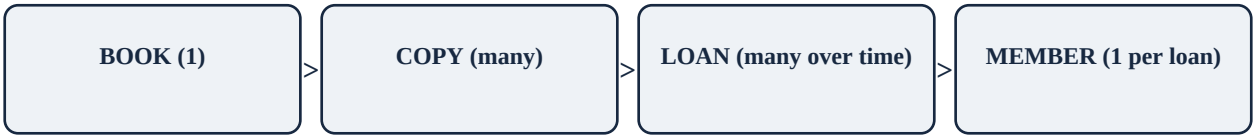


Figure 6. Simplified conceptual entity relationship chain.

18. Book and Copy Data Dictionary

A data dictionary defines what every field means and prevents ambiguous names. A book ID identifies one catalogue description; a copy ID identifies one loanable physical item. For a title with three copies, the book ID is shared while the copy IDs differ.

Required textual fields should reject empty input after trimming whitespace. ID formats can be simple short strings, but they must be unique within the relevant table. A category is useful for filtering; it need not determine whether a book is available.

A removed or damaged copy may need an inactive state distinct from “on loan.” A future implementation could add a copy condition field and exclude inactive copies from availability counts.

Entity.field	Meaning	Example
Book.book_id	Catalogue ID	B01
Book.title	Displayed title	Python Basics
Book.author	Author name	A. Rao
Copy.copy_id	Physical copy ID	C101
Copy.book_id	Parent title ID	B01

Table 8. Book and copy fields; names are fictional.

19. Member and Loan Data Dictionary

The member ID is a stable identifier used in transactions. A real system would require a privacy policy for names and contact details. The case-study examples use fictitious IDs and avoid storing actual phone numbers or addresses.

A loan ID uniquely identifies each transaction. Issue date and due date are recorded when the loan is opened. Return date is empty while the loan is active and filled when the copy comes back. The design must avoid silently overwriting an earlier transaction.

A returned copy can later be issued again with a new loan ID. The earlier loan remains closed in historical records, allowing the current status to be derived without losing the past.

Entity.field	Meaning	Example
Member.member_id	Borrower ID	M01
Loan.loan_id	Transaction ID	L001
Loan.copy_id	Borrowed copy	C101
Loan.issue_date	Date issued	01 Aug 2026
Loan.due_date	Example due date	15 Aug 2026
Loan.return_date	Blank until return	—

Table 9. Member and loan fields; dates are illustrative.

20. Business Rules

Business rules convert general goals into precise decisions. A copy may be issued only if the copy exists, is usable and has no active loan. The member must exist and meet any local eligibility rule. The operation should not create a partial loan when a required check fails.

A return may close only an existing active loan. If a loan has already been returned, a second return must not change its recorded return date. The due date is stored for later comparison; the report does not define a real fine or payment policy.

For summary purposes, total loanable copies equal available copies plus actively issued copies, excluding retired or unusable copies. This identity is a helpful consistency check.

- Invariant 1: at most one active loan exists for each copy.
- Invariant 2: a closed loan has a return date.
- Invariant 3: changing title details does not change physical copy IDs.

These rules are design requirements, not test results.

21. Issue Workflow

The proposed issue process begins when an operator supplies a member ID and copy ID. The system looks up both records and checks eligibility and availability. If a condition fails, it explains the cause and writes no new loan.

If checks pass, the system allocates a loan ID, records the issue date and calculates a due date under a configured policy. The new record has no return date. The copy then appears unavailable in subsequent searches until the loan is closed.

In a real implementation, validation and saving the loan must be performed together so two near-simultaneous requests cannot both issue the same copy. This is particularly important if more than one operator is later supported.



Figure 7. Issue decision flow, shown at a conceptual level.

22. Return Workflow

For a return, the operator enters the copy ID or the active loan ID. The system finds the currently open loan. If no active loan exists, it reports that the item cannot be returned through this operation. This prevents a false change to availability.

When an open loan is found, the operator records the actual return date. The program would compare it with the due date to indicate whether the example is on time or late. Any charge calculation requires a separately approved local policy.

After a successful return, the active-loan lookup finds no open loan for that copy, so the copy can be issued again. The closed record remains available for historical review.

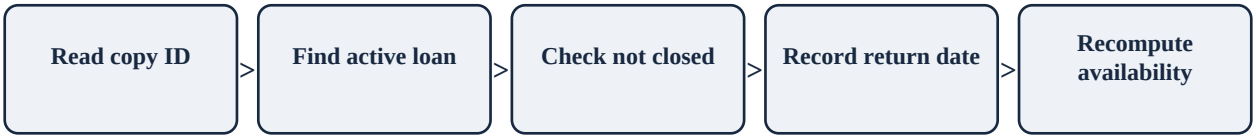


Figure 8. Return decision flow.

23. Search and Catalogue Workflow

A simple catalogue search can accept part of a title, an author name or a category. The text is normalised, for example by trimming spaces and comparing without case sensitivity. The system then presents matching titles with their copy counts.

A result should distinguish total copies from currently available copies. A user can then tell whether the title exists but all copies are borrowed. A match on the title alone is not enough to promise an issue.

For a small dataset a sequential scan is adequate to explain the method. For larger collections, indexing or a database query could improve efficiency, but this report makes no measured speed claim.

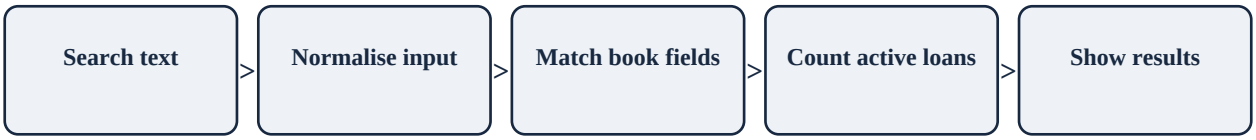


Figure 9. Proposed search process.

24. Availability Calculation

Availability is evaluated per physical copy. Let C be all usable copies of a title and A be those with an open loan. Then available count = |C| - |A|. For example, if a title has three usable copies and one is actively borrowed, two are available.

This arithmetic only works if each active loan refers to an existing copy and the one-active-loan-per-copy rule holds. It should also exclude retired and damaged copies from C. The sample dataset later uses only usable copies to keep the calculation clear.

If availability is calculated from loans each time, it cannot drift from a separately edited status flag. A future implementation may cache a count for speed, but must then maintain it consistently.

Example title	Usable copies	Open loans	Available
Python Basics	3	1	2
Algorithms 101	2	2	0
Database Intro	1	0	1

Table 10. Invented availability example, not a measured library record.

25. Validation and Error Handling

A correct process needs both a successful path and useful failure messages. The operator should not be able to issue an unknown copy, enter a blank member ID or return an item with no open loan. Each failure should leave the stored records unchanged.

Dates also require validation. A return date before the issue date is inconsistent. A due date should not precede the issue date. Duplicate loan IDs must be rejected. If persistent storage is later added, file or database errors must be reported rather than mistaken for successful updates.

Error messages should identify the action the operator can take, such as checking an ID or confirming that a previous return was entered. They should avoid unnecessarily exposing personal information.

Invalid input or state	Expected response
Unknown member ID	Reject issue; request correct ID
Copy already on loan	Reject duplicate issue
No active loan	Reject return; review history
Return before issue	Reject inconsistent date

Table 11. Example validation responses.

26. Privacy and Basic Security

Even a small library can hold personal data. A real implementation should collect only information necessary to manage borrowing, restrict who may access it and define how long it is retained. The illustrative dataset in this report uses fictional member IDs instead of actual borrower identities.

An authorised librarian should be able to edit circulation records; a general viewer should not be able to change them. If passwords are used in a future product, they must not be stored as plain text. A backup plan and an audit trail are also important if records have operational value.

This design does not access EWB Courses systems or data. No internal or restricted information appears in its examples. If an NDA applies to any real internship activity, its evidence and terms must be checked separately before adding such details.

- Minimise personal information.
- Restrict edits to authorised users.
- Back up data before real-world use.
- Use fictional values in public examples.

27. Illustrative Dataset

The following miniature catalogue is invented to show how the proposed rules interact. It does not represent a library, an EWB Courses dataset or an output produced by a Python application. Every copy label is unique.

The title “Python Basics” has three copies; “Algorithms 101” has two; “Database Intro” has one. For clarity, the examples use two fictional member IDs, M01 and M02. Titles and names are placeholders rather than bibliographic citations.

The dataset is deliberately small enough for a reader to check the counts by hand. It is only a demonstration of the design, not a statistical survey.

Book ID	Sample title	Copy IDs	No. of copies
B01	Python Basics	C101, C102, C103	3
B02	Algorithms 101	C201, C202	2
B03	Database Intro	C301	1

Table 12. Fictional catalogue for the worked example.

28. Case Study Scenario

Consider a hypothetical campus reading room that has the six physical copies listed on the previous page. Two fictional members request books. The operator wants to see whether the catalogue and loan register stay consistent through a sequence of issue and return actions.

The scenario begins with no open loans. On 1 August 2026, C101 is issued to M01 with an illustrative fourteen-day due period. On the same day, C201 and C202 are issued to M02 and M01 respectively. All three copies are now unavailable, while the other three remain available.

A later return of C101 closes its loan. Attempting to issue C201 again before it is returned must fail, while issuing C102 would remain possible. These outcomes follow the rules; they are not execution results.

Stage	Active loans	Available copies
Initial	0	6
After three illustrative issues	3	3
After returning C101	2	4

Table 13. Manually derived scenario totals.

29. Worked Example: First Issue

Assume M01 is registered and C101 exists as a usable copy of B01. The open-loan search initially finds no record for C101. The proposed issue operation therefore passes validation and creates loan L001 linking member M01 to copy C101.

If the issue date is 1 August 2026 and the example policy is fourteen days, the due date is 15 August 2026. The return date remains empty. As a consequence, Python Basics has two available copies, C102 and C103, even though its catalogue record still lists three total copies.

This is a hand-worked state transition. It demonstrates what a future program should do; it does not document a screen or test run.

Field	Before issue	After issue
Active loan for C101	None	L001
Available B01 copies	3	2
L001 return date	Not applicable	Blank

Table 14. Predicted state change for the first issue.

30. Worked Example: Duplicate Issue

Now suppose a second request tries to issue C101 while L001 has no return date. The system would find an active loan for that copy and reject the request. It must not create a second open loan or silently reassign L001 to another member.

A different physical copy of the same title, C102, remains available. This distinction is the reason the design models a Book and its Copies separately. Rejecting C101 does not imply rejecting every copy of B01.

The proposed rejection message could be: “Copy C101 is currently on loan; choose an available copy.” It should be specific enough to help the operator, without revealing the borrower’s identity to unauthorised users.

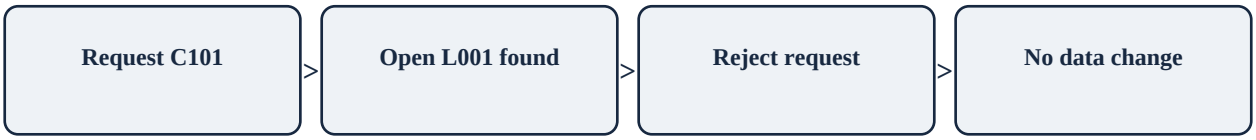


Figure 10. Duplicate-issue prevention in the worked example.

31. Worked Example: Return

On 10 August 2026 the operator records the return of C101 against active loan L001. The date is after the illustrative issue date and before the illustrative due date, so the case is on time under the assumed fourteen-day rule.

The return date is added to L001, which remains in the loan history. The active-loan search for C101 now finds no open loan. The number of available copies of Python Basics rises from two to three unless another loan is created afterward.

If the operator tries to return C101 again without a new issue, the system should reject the duplicate action. This protects the original historical return date from accidental overwrite.

Field	Before return	After return
L001 return date	Blank	10 Aug 2026
C101 availability	Unavailable	Available
Available B01 copies	2	3

Table 15. Predicted return state; example dates only.

32. Worked Example: Search

Searching for “python” should match the title Python Basics regardless of letter case if a case-insensitive comparison is used. The result should show the book and its three copy IDs. Before returning C101, it would show two available copies; afterward, it would show three.

Searching for a title not in the sample catalogue should return an empty result rather than an invented record. A blank query could either be rejected or interpreted as “show all books”; the choice should be documented before implementation.

A user-facing search result should communicate both presence and loanability. A title with zero available copies still belongs in the catalogue and may be useful for a future reservation feature.

Query	Illustrative match	Interpretation
python	B01 / Python Basics	Case-insensitive title match
unknown title	None	No matching catalogue record
Algorithms	B02 / Algorithms 101	May have zero available copies

Table 16. Expected search behaviour, not an application screenshot.

33. Summary Reporting

A librarian may want a small snapshot of collection activity. Total titles count Book records; total copies count usable Copy records; active loans count Loan records with no return date; available copies are usable copies minus active loans.

For the illustrative dataset, the initial snapshot has three titles, six copies, zero active loans and six available copies. After three example issues, active loans increase to three and available copies decrease to three. After one return, the active-loan count becomes two and availability becomes four.

Counts should be described as a point-in-time view. They can change after another issue, return or withdrawal. The values here are derived manually from the fictional transactions, not from a deployed reporting module.

Metric	Initial	After 3 issues	After 1 return
Book titles	3	3	3
Usable copies	6	6	6
Active loans	0	3	2
Available copies	6	3	4

Table 17. Manually calculated example summary.

34. Charts and Interpretation

The bar chart below visualises the invented scenario at three stages. Each stage contains the same six usable copies. The “available” portion decreases when three loans are opened and increases when one loan is returned. The chart is a diagram of the rules, not a measured performance result.

A chart can make a logical state change easier to explain during a viva. It cannot prove that software works. Evidence of correct implementation would require code, test results and real execution, none of which is claimed in this theory-only report.

The invariant is visible in every bar: available copies plus active loans equals six. If a future application produced a different total, its stored records would need investigation.

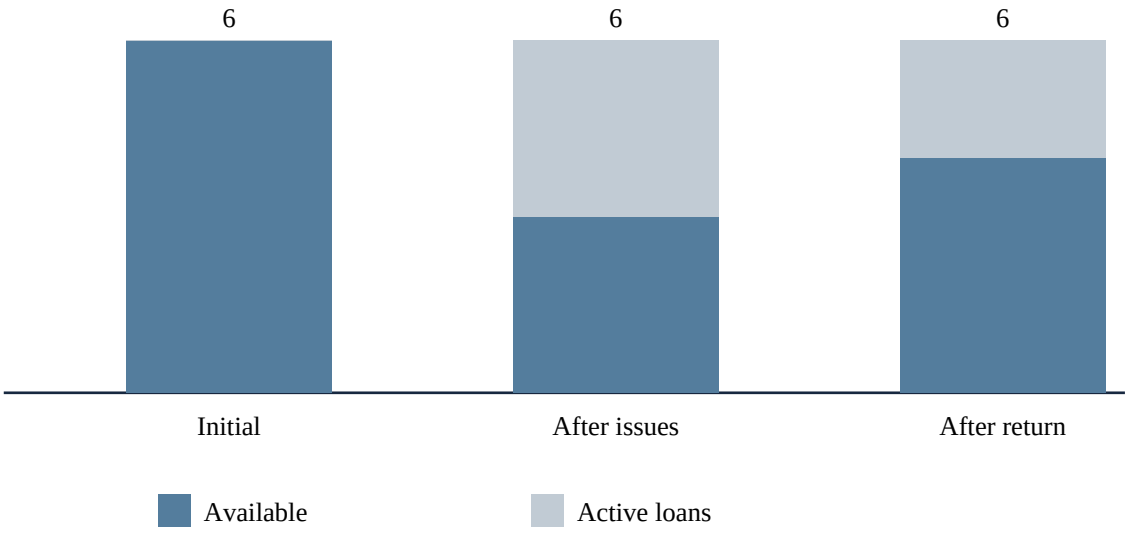


Figure 11. Illustrative available/issued counts; invented values.

35. Proposed Test Plan

Testing would compare a future implementation against the requirements before using it with real records. For each case, the tester would prepare a known initial state, perform one operation and inspect the expected state and message.

A useful test plan includes normal paths, invalid IDs, boundary conditions and repeated actions. Tests must also verify that a rejected operation does not alter the loan register. A good design is not confirmed merely because one successful example can be described.

The following page lists expected outcomes only. No automated tests were run for this report and no pass rate is reported.

- Prepare a small fictional dataset and reset it between tests.
- State the expected result before performing each future test.
- Check both message and stored-record changes.
- Document any failed behaviour before release.

36. Proposed Test Cases

A future programmer can use the cases below as acceptance checks. The normal issue and return cases exercise the expected path; the duplicate and unknown-ID cases test validation. The results column states the predicted behaviour of the design, not observed software output.

Where a test says “unchanged,” the catalogue and loan register should be exactly as they were before the rejected action. A real test should verify record contents, not only the displayed message.

Case	Input / condition	Expected result
T1	Issue available C101 to M01	Create one active loan
T2	Issue C101 again before return	Reject; no new loan
T3	Return C101 against L001	Close loan; copy available
T4	Return C101 a second time	Reject; history unchanged
T5	Issue unknown C999	Reject; no new loan
T6	Search title with mixed case	Same title match

Table 18. Planned tests; none are claimed as executed.

37. Design Limitations

This study does not contain source code, an interface, user feedback, execution logs or deployment evidence. Its tables and figures illustrate logical outcomes only. It therefore cannot demonstrate usability, actual speed, fault tolerance or compliance with a particular library policy.

The design assumes a small single-location library. In a real system, concurrent operators, barcode labels, damaged copies, reservations, renewals and member eligibility rules would require more detailed treatment. Real data also brings privacy and backup obligations.

Some details are intentionally open: the accepted ID format, real loan duration, maximum loans per member and fine policy must be set by the library. The sample fourteen-day period is only a worked-example assumption.

Honest limitation: a theory-only design is not a functioning application.

38. Future Scope

If the conceptual design is approved, a small command-line implementation could first create the four record types and the issue, return and search functions. An SQLite database could preserve records and enforce unique IDs. Test cases in this report could then be implemented and run.

Later additions might include reservations, renewal rules, overdue reminders and a dashboard. These should follow an agreed institutional policy. A barcode scanner or web interface would not replace the need for correct copy-level loan rules.

A measured evaluation should be conducted only after a real implementation exists. It could compare record accuracy and task completion time with an agreed baseline while protecting personal data.

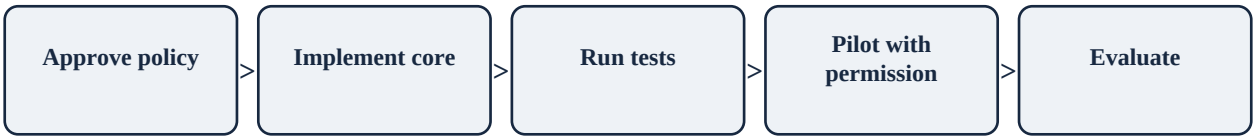


Figure 12. Possible future development sequence, not work completed.

39. Conclusion

This report has described a beginner-friendly, Python-oriented library management design using four related record types: Book, Copy, Member and Loan. Its key idea is that availability belongs to each physical copy and can be determined from whether an active loan exists.

The illustrative case study showed how an issue creates an open loan, how a duplicate request is rejected, and how a return closes the loan. Manually calculated tables and a chart make these rules visible without presenting them as software output.

The design is suitable as a theoretical application of Python programming concepts. It must be distinguished from the student’s documented internship completion: the attached certificate confirms Python Programming training, while this report’s library scenario is an academic example rather than a verified employer deliverable.

End of main report.

REFERENCES

1. Python Software Foundation. “The Python Tutorial.” https://docs.python.org/3/tutorial/ (access date to be added by student).

2. Python Software Foundation. “Data Structures.” https://docs.python.org/3/tutorial/datastructures.html (access date to be added by student).

3. Python Software Foundation. “sqlite3 — DB-API 2.0 interface for SQLite databases.” https://docs.python.org/3/library/sqlite3.html (access date to be added by student).

4. Python Software Foundation. “datetime — Basic date and time types.” https://docs.python.org/3/library/datetime.html (access date to be added by student).

5. University of Lucknow, Department of Computer Science and Engineering. Supplied Industrial Training Report cover-page template (student-provided PDF).

6. EWB EDUTECH PRIVATE LIMITED. Internship Offer Letter addressed to Mayank Kumar, dated 14 June 2026 (student-provided document).

7. EWB Courses. Internship Completion Certificate awarded to Mayank Kumar, issued 15 July 2026 (student-provided document).

APPENDIX A: GLOSSARY

Book: Catalogue description of a title or edition.

Copy: One physically loanable item belonging to a book record.

Member: Registered person to whom a copy may be issued.

Loan: Transaction that links a member and copy over a period of time.

Active loan: Loan with no recorded return date.

Availability: Whether a usable copy has no active loan.

Validation: Checking input and current state before updating records.

Conceptual design: Explanation of proposed behaviour without claiming an implementation.

APPENDIX B: SUBMISSION CHECKLIST

[ ] Fill in the roll number, section, declaration date and signature on the printed report.

[ ] Confirm that the certificate is accepted as evidence of the required four-to-six-week duration; the supplied documents say “one month” but do not specify exact start and end dates.

[ ] If the department asks for a specific internship project, verify whether this separate academic case study is acceptable. The supplied certificate does not name this library project.

[ ] Add genuine details of actual training activities only if personally verified. Do not describe the diagrams or expected tests as screenshots of a running model.

[ ] Print all pages on one side only, use a colour print for the certificate page and spiral-bind the report.

[ ] Submit by 16 November 2026: Section 1 to Dr. Zeeshan Ali Siddiqui; Section 2 to Dr. Sakshi Srivastava. Confirm your section before submission.